

Yenepoya (Deemed to be University)

Project – High Level Design

on

AI Self-Healing Cloud for Retail

Student Name:

Mohammed Raed (23BCAICD064)

Industry Mentor: _____, **Guided by:** [Guide
Name]

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Scope Of The Document

1.2 Intended Audience

2. SYSTEM OVERVIEW

3. SYSTEM DESIGN

3.1 Application Design

3.2 Process Flow

3.3 Information Flow

3.4 Components Design

3.5 Key Design Considerations

4. API CATALOGUE

5. DATA DESIGN

5.1 Data Model

5.2 Data Access Mechanism

5.3 Data Retention Policies

5.4 Data Migration

6. INTERFACES

7. STATE AND SESSION MANAGEMENT

8. CACHING

9. NON-FUNCTIONAL REQUIREMENTS

9.1 Security Aspects

9.2 Performance Aspects

10. REFERENCES

1. INTRODUCTION

1.1 Scope Of The Document

This document establishes the high-level architecture for the AI Self-Healing Cloud system. It delineates system integration points, macro-level data flow, and the methodologies powering an autonomous Service Reliability Engineering (SRE) retail environment. The scope encompasses operating system-level process monitoring, self-healing execution, relational database persistence, and low-latency telemetry rendering. Furthermore, it serves as the authoritative blueprint demonstrating how the infrastructure achieves zero-downtime operations without manual human intervention.

1.2 Intended Audience

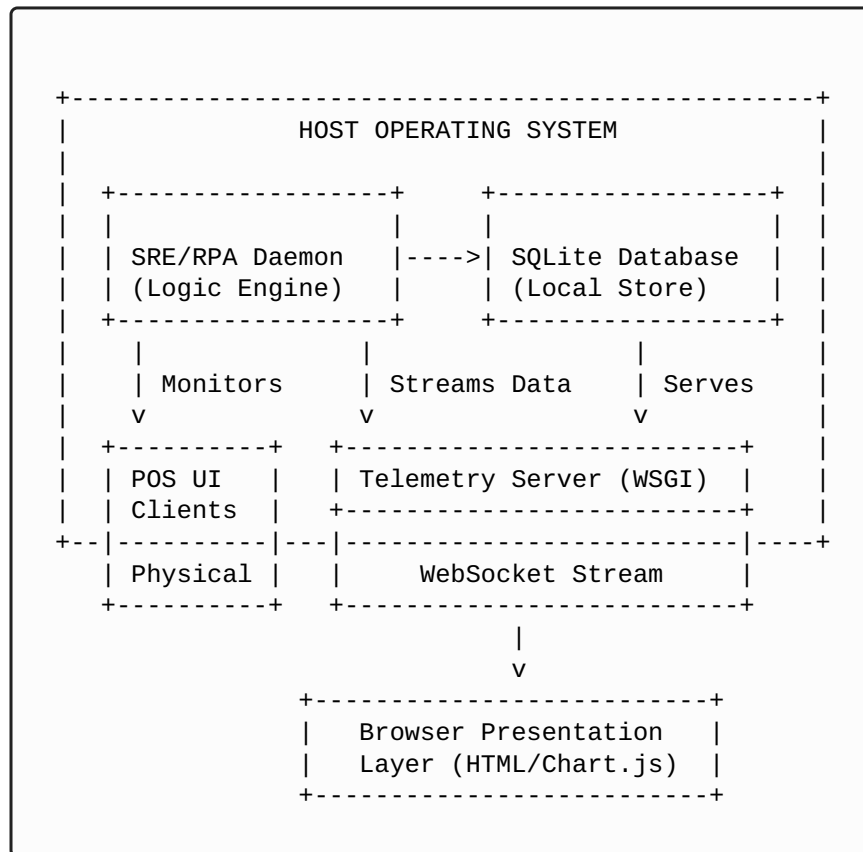
This document is authored for university project evaluators, faculty review committees, industry mentors, and enterprise systems architects. It provides the architectural abstractions required to assess structural validity, fault tolerance, and software readiness. It assumes foundational knowledge of client-server topologies, multithreaded daemon design, and asynchronous telemetry architectures.

2. SYSTEM OVERVIEW

The AI Self-Healing Cloud represents a highly available, fault-tolerant kiosk ecosystem designed to secure autonomous commercial checkout environments. Traditional Point-of-Sale (POS) systems require manual intervention during critical application failures, leading to revenue loss. This system mitigates this risk by deploying a headless background daemon that actively polls host operating system process tables.

Upon detecting a failure in monitored client applications, the daemon instantly executes a deterministic restart protocol, ensuring absolute operational continuity. Concurrently, a stochastic automation engine synthesizes transaction data, drives the desktop User Interface (UI), and archives records in an ACID-compliant SQLite database. A locally-hosted web dashboard ingests event-driven WebSockets to display live metrics, including aggregate revenue and active system health.

Figure 2.1: Macro System Architecture and Context



3. SYSTEM DESIGN

3.1 Application Design

The system is engineered upon an Asynchronous Multi-Threaded Hybrid Architecture. To ensure the centralized web server remains responsive to HTTP requests while the automation engine performs hardware-blocking I/O operations, execution spans two primary threads. The primary thread operates the WSGI Telemetry Server for uninterrupted full-duplex communication. The daemonized thread manages OS process health checks, transaction generation, and physical UI manipulation.

3.2 Process Flow

The operational lifecycle adheres to a strict pipeline. It initiates with a Boot Phase mounting the database and binding the WebSocket server. The Telemetry Polling phase scans the Process Table for target executables. If a breach is detected, the Self-Healing Trigger executes recovery protocols. Upon stabilization, Stochastic Generation synthesizes parameters. These enter the Persistence phase, committing payloads to the local database, before Event Emission broadcasts the updated state to the presentation layer.

3.3 Information Flow

Hardware interrupts and process states flow upward from the OS Kernel directly to the SRE Engine. Synthesized transaction data follows a bifurcated downstream flow: it is simultaneously written to the physical disk via the SQLite engine for immutable storage, and pushed outward via WebSocket TCP streams to the volatile Document Object Model (DOM) of the frontend client for live rendering.

3.4 Components Design

The system is compartmentalized into four modules. The SRE Engine provides autonomous lifecycle management. The RPA Engine provides peripheral emulation for raw hardware commands. The Telemetry Backend serves as the networking bridge, utilizing a Flask server. Finally, the Live Dashboard acts as the decoupled presentation layer, utilizing CSS grids and charting libraries.

3.5 Key Design Considerations

A paramount architectural consideration was mitigating race conditions—specifically OS-level window focus delays during physical automation—addressed through deliberate thread sleep cycles. Database operations are strictly headless and scoped; connections are instantiated and terminated within a single transaction lifecycle to prevent operational locks during rapid iterations.

4. API CATALOGUE

The application diverges from traditional polling-heavy REST architectures by implementing a low-latency, event-driven communication framework. This design choice fundamentally minimizes network overhead while ensuring absolute real-time synchronization between the internal daemon state and the external visual dashboard. By decoupling the static asset delivery from the dynamic data pipeline, the architecture achieves a highly optimized footprint suitable for resource-constrained kiosk environments.

The primary initialization endpoint operates on the root route ("/"). This interface utilizes standard HTTP GET methodologies. Its sole architectural purpose is to serve the compiled HTML, CSS, and JavaScript assets to the client browser upon the initial connection request. Once the browser parses this payload, it automatically initiates the client-side socket handshake. Following this successful handshake, standard HTTP traffic effectively ceases.

The core telemetry is transmitted via the "update_data" WebSocket event. This is a unidirectional Server-to-Client push mechanism. The transmitted payload is a serialized JSON object containing critical state variables: total transactions, items, aggregate revenue, and system status markers. Because the server pushes this data natively upon mutation, the client dashboard never executes expensive database polling queries. Finally, an administrative interface operates on the "/download_report" route, querying historical records and returning a direct CSV file stream via HTTP GET.

5. DATA DESIGN

5.1 Data Model

The system relies on a localized, relational data model managed entirely within SQLite3. The schema revolves around a primary "transactions" table encompassing a unique transaction identifier (Primary Key), a string-based customer representation, integer-based item quantities, floating-point currency representations for unit price and total cost, and an ISO-formatted timestamp.

5.2 Data Access Mechanism

Data access and manipulation are executed directly via native database drivers. The system strictly enforces the use of parameterized SQL queries for all data insertion methodologies. This architectural mandate actively neutralizes potential SQL injection vulnerabilities by decoupling the executable SQL syntax from the synthesized data payload.

5.3 Data Retention Policies

Transactional records are stored persistently on the local disk. By design, this data remains immutable and intact across system reboots, hardware power failures, or application crashes. The system functions as a permanent operational ledger and does not implement automated cron-based data purging or truncation.

5.4 Data Migration

To facilitate broader enterprise integration, data extraction is abstracted through a dedicated export utility. This utility parses the proprietary binary database format and translates the historical ledger into a universally compatible comma-separated values (CSV) format, ensuring seamless ingestion into external auditing frameworks.

6. INTERFACES

The architecture dictates three primary integration boundaries bridging headless background code execution with physical retail workflows. These boundaries are heavily compartmentalized to ensure that a failure in one interface does not cascade and corrupt the underlying logic engine or database integrity.

The User Interface (UI) is a decoupled, browser-based dashboard hosted locally on port 5001. This interface provides a read-only, real-time command center view of the kiosk's operations. It utilizes CSS grid layouts to render raw integer data dynamically and integrates the Chart.js library to produce animated graphical canvases representing revenue growth over time. Because it relies purely on ingested WebSocket data, the UI is completely isolated from the automation logic.

The System Interface involves programmatic hooking into standard Windows OS executables. The Python engine interfaces directly with the Windows API to launch, monitor, and manipulate the window states of target binaries. This requires the application to aggressively steal window focus, bypass OS-level sleep states, and inject raw hardware keystrokes. Lastly, the Alerting Interface dispatches outbound HTTP POST payloads to a remote Webhook API to provide critical failure notifications to human engineers.

7. STATE AND SESSION MANAGEMENT

In traditional web applications, state and session management require complex infrastructures involving Redis caching, JWT tokens, or persistent browser cookies to track user behavior across multiple pageloads. The AI Self-Healing Cloud fundamentally bypasses this complexity by maintaining the application state globally within the backend Python application layer using a localized, strongly-typed memory dictionary.

This specific data structure serves as the single source of truth for the active execution session. It accurately tracks cumulative revenue, total item counts, and localized shift metrics across the entire runtime of the script. As transaction variables are generated and processed during the automation loop, this dictionary is mutated in-place. This ensures the state is instantly updated in RAM prior to any WebSocket emission.

Because the system is designed strictly as a localized kiosk monitor rather than a globally distributed web application, the dashboard state is entirely ephemeral regarding the web view. If a user closes and reloads the browser, the frontend simply connects to the socket and fetches the latest push from the dictionary. However, the transactional truth is permanently backed by the SQLite database, ensuring perfect synchronization between physical operations and digital telemetry.

8. CACHING

To optimize overall system performance, prevent CPU bottlenecking, and vastly reduce disk I/O latency, the system implements localized caching protocols at both the backend processing and frontend rendering layers. Without caching, the system would be forced to execute expensive database aggregations during every loop cycle to update the dashboard, leading to severe operational degradation over an active shift.

On the backend, aggregated shift metrics (such as total gross revenue and total items sold) are mathematically accumulated and cached directly within the active Python thread memory. By mutating these variables in-place during the RPA loop, the system completely prevents expensive SQL SUM() or COUNT() queries from executing against the database. This strategy ensures the SQLite database is strictly reserved for lightweight, append-only operations.

On the presentation layer, real-time line graph plotting data is cached locally within the client-side browser using native JavaScript arrays. As new WebSocket payloads arrive, the data is pushed into the Chart.js instance arrays. To ensure the browser does not suffer from memory leaks or visual degradation during extended operation, an algorithmic garbage collection check is implemented. This algorithm monitors the array length and dynamically shifts out the oldest data points once a specific threshold is reached.

9. NON-FUNCTIONAL REQUIREMENTS

9.1 Security Aspects

The application enforces strict network security postures. The telemetry server is programmatically restricted to bind exclusively to the localhost loopback interface (127.0.0.1). This constraint categorically denies unauthorized external network agents from accessing the telemetry dashboard or triggering administrative export endpoints.

9.2 Performance Aspects

The system is algorithmically tuned for continuous, high-performance runtime. The utilization of WebSockets establishes an open TCP connection, entirely circumventing HTTP header parsing latency to achieve sub-100ms dashboard telemetry updates. UI interaction intervals are artificially throttled to safely align with hardware rendering constraints.

10. REFERENCES

1. Flask Web Development Framework Architecture: <https://flask.palletsprojects.com/>
2. SQLite3 Relational Database Engine Documentation: <https://www.sqlite.org/docs.html>
3. WebSocket Protocol Specification (RFC 6455): <https://datatracker.ietf.org/doc/html/rfc6455>